

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

from datasets import load_dataset
import numpy as np
import torchvision.transforms as transforms
```

```
dataset = load_dataset("ylecun/mnist")

train_ds = dataset["train"]
test_ds = dataset["test"]
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your setti
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to ac
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
README.md: 6.97k/? [00:00<00:00, 213kB/s]

mnist/train-00000-of- 15.6M/15.6M [00:01<00:00, 16.0MB/s]
00001.parquet: 100%

mnist/test-00000-of- 2.60M/2.60M [00:00<00:00, 7.52MB/s]
00001.parquet: 100%

Generating train split: 100% 60000/60000 [00:00<00:00, 142131.88 examples/s]
```

```
def transform(example):
    image = np.array(example["image"]) / 255.0
    image = torch.tensor(image, dtype=torch.float32).unsqueeze(0)
    label = torch.tensor(example["label"], dtype=torch.long)
    return {"image": image, "label": label}

train_ds = train_ds.map(transform)
test_ds = test_ds.map(transform)

train_ds.set_format(type="torch", columns=["image", "label"])
test_ds.set_format(type="torch", columns=["image", "label"])
```

```
Map: 100% ██████████ 60000/60000 [00:38<00:00, 2029.98 examples/s]
Map: 100% ██████████ 10000/10000 [00:07<00:00, 1898.79 examples/s]
```

```
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=1000, shuffle=False)
```

```
augment = transforms.Compose([
    transforms.RandomRotation(10),
    transforms.RandomAffine(0, translate=(0.1, 0.1))
])
```

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.net(x)
```

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv(x)
        return self.fc(x)
```

```
class TransformerModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed_dim = 64

        self.patch_embed = nn.Conv2d(1, self.embed_dim, kernel_size=7,
                                       stride=2, padding=2)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=self.embed_dim,
            nhead=4,
            batch_first=True
        )

        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=6)
        self.fc = nn.Linear(self.embed_dim, 10)

    def forward(self, x):
        x = self.patch_embed(x)
        x = x.flatten(2).permute(0, 2, 1)

        x = self.transformer(x)
        x = x.mean(dim=1)

        return self.fc(x)
```

```
def train(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0

    for batch in loader:
        data = batch["image"]
        target = batch["label"]

        data = torch.stack([augment(img) for img in data])

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    return total_loss / len(loader)
```

```
def test(model, loader):
    model.eval()
    correct = 0

    with torch.no_grad():
        for batch in loader:
            data = batch["image"]
            target = batch["label"]

            output = model(data)
            pred = output.argmax(dim=1)
            correct += (pred == target).sum().item()

    return 100 * correct / len(loader.dataset)
```

```
print("\nTraining CNN Model...")
model = CNN()
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

cnn_acc = 0

for epoch in range(5):
    loss = train(model, train_loader, optimizer, criterion)
    cnn_acc = test(model, test_loader)
    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={cnn_acc:.2f}%")
```

```
Training CNN Model...
Epoch 1: Loss=0.4082, Acc=98.09%
Epoch 2: Loss=0.1571, Acc=98.55%
Epoch 3: Loss=0.1226, Acc=98.66%
Epoch 4: Loss=0.1001, Acc=99.15%
Epoch 5: Loss=0.0892, Acc=99.01%
```

```
print("\nTraining MLP Model...")
model = MLP()
optimizer = optim.Adam(model.parameters(), lr=0.001)

mlp_acc = 0

for epoch in range(5):
    loss = train(model, train_loader, optimizer, criterion)
    mlp_acc = test(model, test_loader)
    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={mlp_acc:.2f}%")
```

```
Training MLP Model...
Epoch 1: Loss=0.6580, Acc=94.30%
Epoch 2: Loss=0.2727, Acc=95.78%
Epoch 3: Loss=0.2164, Acc=96.62%
Epoch 4: Loss=0.1846, Acc=96.20%
Epoch 5: Loss=0.1660, Acc=97.33%
```

```
print("\nTraining Transformer Model...")
model = TransformerModel()
optimizer = optim.Adam(model.parameters(), lr=0.001)

trans_acc = 0

for epoch in range(5):
    loss = train(model, train_loader, optimizer, criterion)
    trans_acc = test(model, test_loader)
    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Acc={trans_acc:.2f}%")
```

```
Training Transformer Model...
Epoch 1: Loss=1.2027, Acc=81.28%
Epoch 2: Loss=0.6546, Acc=88.13%
Epoch 3: Loss=0.5167, Acc=90.27%
Epoch 4: Loss=0.4413, Acc=91.97%
Epoch 5: Loss=0.4007, Acc=91.79%
```

```
print("\n=== Final Model Comparison ===")
print(f"MLP Accuracy          : {mlp_acc:.2f}%")
print(f"CNN Accuracy          : {cnn_acc:.2f}%")
print(f"Transformer Accuracy: {trans_acc:.2f}%")
```

```
=== Final Model Comparison ===
MLP Accuracy          : 97.33%
CNN Accuracy          : 99.01%
Transformer Accuracy: 91.79%
```

